

TRABAJO PRÁCTICO INTEGRADOR 2026

Materia: Sistemas Distribuidos
Licenciatura en Ciencias de la Computación

1. OBJETIVO DEL TRABAJO

El objetivo de este trabajo es diseñar, implementar y desplegar una aplicación distribuida que utilice un modelo de comunicación asíncrona mediante una cola de mensajes (*Message Broker*). Se busca que el estudiante demuestre capacidad para desacoplar componentes, gestionar la comunicación entre servicios y realizar el despliegue en un entorno de orquestación (Kubernetes).

2. CONSIGNA TÉCNICA

El grupo deberá desarrollar una aplicación compuesta por componentes distribuidos que interactúen de la siguiente manera:

1. **Cliente (Frontend Interfaz de Usuario):** Una aplicación web/GUI/TUI/CLI que permita al usuario interactuar con el sistema.
2. **Backend (API Gateway/Service):** Un servicio que reciba las peticiones del cliente mediante protocolos estándar (HTTP/REST) y actúe como productor de mensajes.
3. **Message Broker:** Un servicio de mensajería (ej. Redis, RabbitMQ, NATS, Kafka, etc) que actúe como intermediario de comunicación.
4. **Worker (Procesador):** Uno o más servicios que consuman los mensajes de la cola, realicen una tarea (procesamiento de datos, simulación de carga, etc.) y devuelvan un resultado o actualicen un estado.

 **REGLA CRÍTICA DE DISEÑO Y SEGURIDAD:** Bajo ninguna circunstancia el cliente (browser del usuario o aplicación externa) podrá conectarse directamente al *Message Broker*. Toda comunicación entre la capa de presentación y la lógica de negocio debe pasar obligatoriamente por una API controlada.

Opcionales

Los siguientes puntos son OPCIONALES y serán considerados como extras que suman valor en el puntaje evaluativo del proyecto.

- **Observabilidad (o11y):** Instrumentación de trazabilidad distribuida con OpenTelemetry o métricas con Prometheus/Grafana para visualizar flujos de mensajes y latencias en dashboards en tiempo real.
- **Seguridad (DevSecOps):** Auditoría de vulnerabilidades en imágenes Docker con Trivy y aplicación de Kubernetes Network Policies para restringir el tráfico interno siguiendo el principio de menor privilegio.
- **Servicios Externos:** Integración con APIs de terceros (notificaciones, geolocalización, etc.) para delegar funcionalidades específicas y enriquecer el

procesamiento asíncrono realizado por los Workers.

- **Dead Letter Queue (DLQ):** Implementación de una cola secundaria para capturar y aislar mensajes fallidos tras múltiples reintentos, permitiendo la inspección técnica de "mensajes venenosos" sin interrumpir el flujo principal del sistema.
- **Auto-escalado por eventos (KEDA):** Implementación de escalado dinámico de Workers basado en la profundidad de la cola de mensajes, permitiendo que la infraestructura de Kubernetes reaccione a la carga real y escale a cero si no hay demanda.
- **Service Mesh:** Despliegue de una malla de servicios (como Istio o Linkerd) para garantizar comunicación cifrada vía mTLS entre componentes y gestionar el tráfico con políticas avanzadas de red.
- **Idempotencia y Consistencia:** Diseño de lógica de procesamiento que garantice la integridad del sistema ante reintentos del broker (entrega at-least-once), asegurando que mensajes duplicados no afecten el estado final de la aplicación.
- **Benchmarking de Carga:** Ejecución de pruebas de estrés con k6 o Locust para identificar cuellos de botella en la infraestructura y documentar analíticamente el límite de saturación de cada microservicio.
- **Despliegues Progresivos:** Configuración de estrategias Canary o Blue-Green mediante el Ingress Controller para realizar actualizaciones de servicios con control de tráfico, minimizando el riesgo en producción.
- **CI/CD:** build automático y posterior deployment usando algún mecanismo como Github Actions, ArgoCD o FluxCD.

3. CALENDARIO DE ENTREGAS PARCIALES

Para garantizar el éxito del proyecto, se establecen tres hitos de entrega obligatorios. El incumplimiento de los hitos afectará la nota final.

Hito	Fecha	Objetivo de la Entrega	Requerimiento Técnico
Hito 1: Diseño y Arquitectura	Jueves 21/05/2026	Validación de la idea y el diseño.	Documento con diagrama de arquitectura (componentes, flujo de mensajes y protocolos) y elección de tecnologías.
Hito 2: PoC Funcional (Local)	Martes 02/06/2026	Validación de la lógica distribuida.	Prueba de concepto con código fuente de comunicación exitosa entre Backend, Broker y Worker (ejecutables mediante Docker Compose o cluster k8s).

Hito	Fecha	Objetivo de la Entrega	Requerimiento Técnico
Hito 3: Entrega Final y Despliegue	09/06/2026 y 11/06/2026 (por sorteo)	Validación de la infraestructura.	Repositorio con manifiestos de Kubernetes (Rke2), imágenes en Registry y documentación completa.

4. REQUERIMIENTOS DE IMPLEMENTACIÓN

- **Lenguajes de Programación:** Libre elección, siempre que se garantice la separación de procesos y servicios.
- **Contenerización:** Cada componente debe contar con su propio **Dockerfile**. Las imágenes deben ser publicadas en un Registry (Docker Hub u otro).
- **Orquestación:** Se debe proveer el conjunto de manifiestos de Kubernetes (Deployments, Services, ConfigMaps/Secrets) para desplegar la arquitectura completa en el cluster **Rke2** de la cátedra.
- **Documentación:** El repositorio debe incluir un archivo **README.md** con:
 - Diagrama de arquitectura (pueden usar Mermaid.js o una imagen).
 - Instrucciones detalladas de despliegue y pruebas.
 - Explicación del flujo de un mensaje desde el cliente hasta el worker.

5. CONDICIONES GENERALES Y ENTREGA

- **Modalidad:** Grupos de hasta 4 (cuatro) personas.
- **Repositorio:** Se debe realizar la entrega a través de un repositorio en GitHub dentro de la organización de la cátedra.
- **Formato del nombre del repositorio:**
PI-<Apellido1>-<Apellido2>-<Apellido3>
- **Fecha límite de entrega final:** Martes 09 y Jueves 11 de Junio de 2026 a las 18:00hs. Dependiendo del sorteo de grupos.
- **Defensa:** Los trabajos serán presentados de forma oral por sorteo que se realizará el día 12 de Mayo de 2026 a las 18hs (en clase).